

# Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: [minhaz.ahmed@gmail.com](mailto:minhaz.ahmed@gmail.com)

# Content

- ▶ Transformer Blocks
- ▶ Use Transformer Block for summarization
- ▶ The input: embeddings for token and position
- ▶ The Language Modeling Head
- ▶ Large Language Models with Transformers
- ▶ Large Language Models: Generation by Sampling
- ▶ Large Language Models: summarization.

# Map coordinates

- ▶ Embeddings: vector representation or meaningful number value
- ▶ Embeddings: similar items have similar lists of numbers
- ▶ Cities that are geographically close (like Tashkent and Samarkand) will have relatively similar coordinates.
- ▶ Cities that are far apart (like Tashkent and New York) will have very different coordinates. Embeddings do similar, but instead of geographic location, they try to capture meaning, context, or relationships in an abstract meaning space.

# Transformer Blocks

**Putting it all together** The function computed by a transformer block can be expressed as:

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{SelfAttention}(\mathbf{X})) \quad (10.24)$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O})) \quad (10.25)$$

Or we can break it down with one equation for each component computation, using  $\mathbf{T}$  (of shape  $[N \times d]$ ) to stand for transformer and superscripts to demarcate each computation inside the block:

$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X}) \quad (10.26)$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1 \quad (10.27)$$

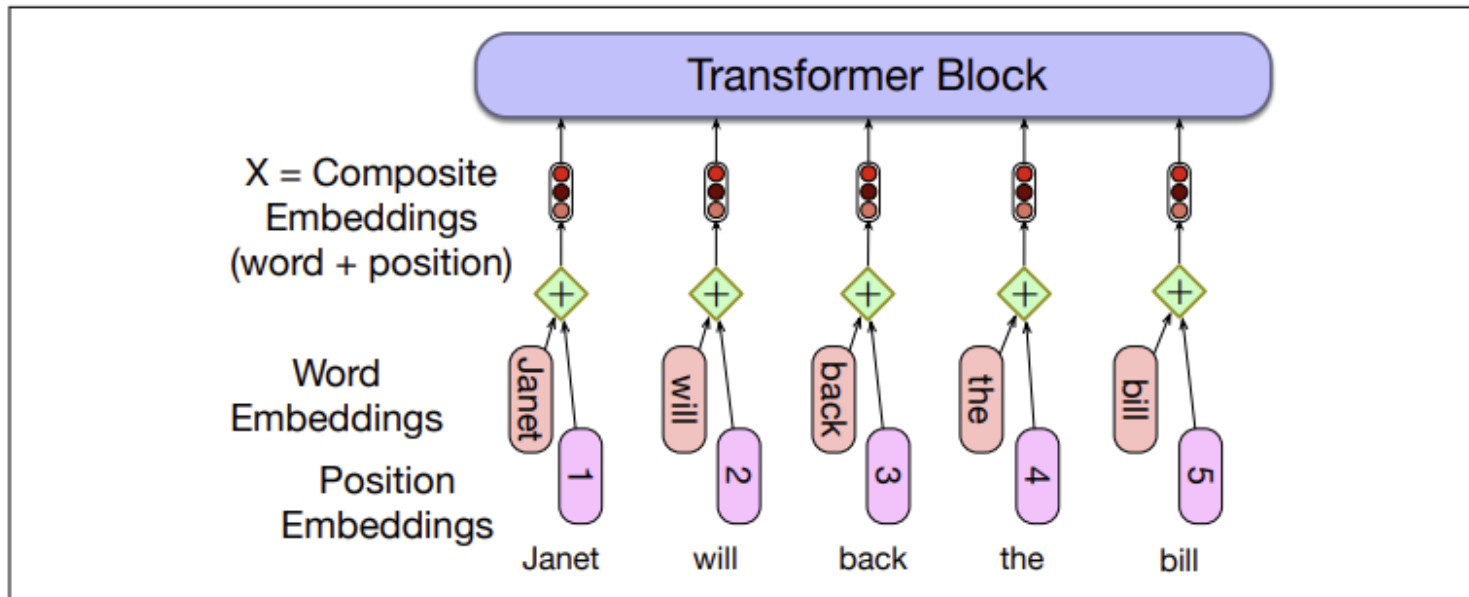
$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2) \quad (10.28)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3) \quad (10.29)$$

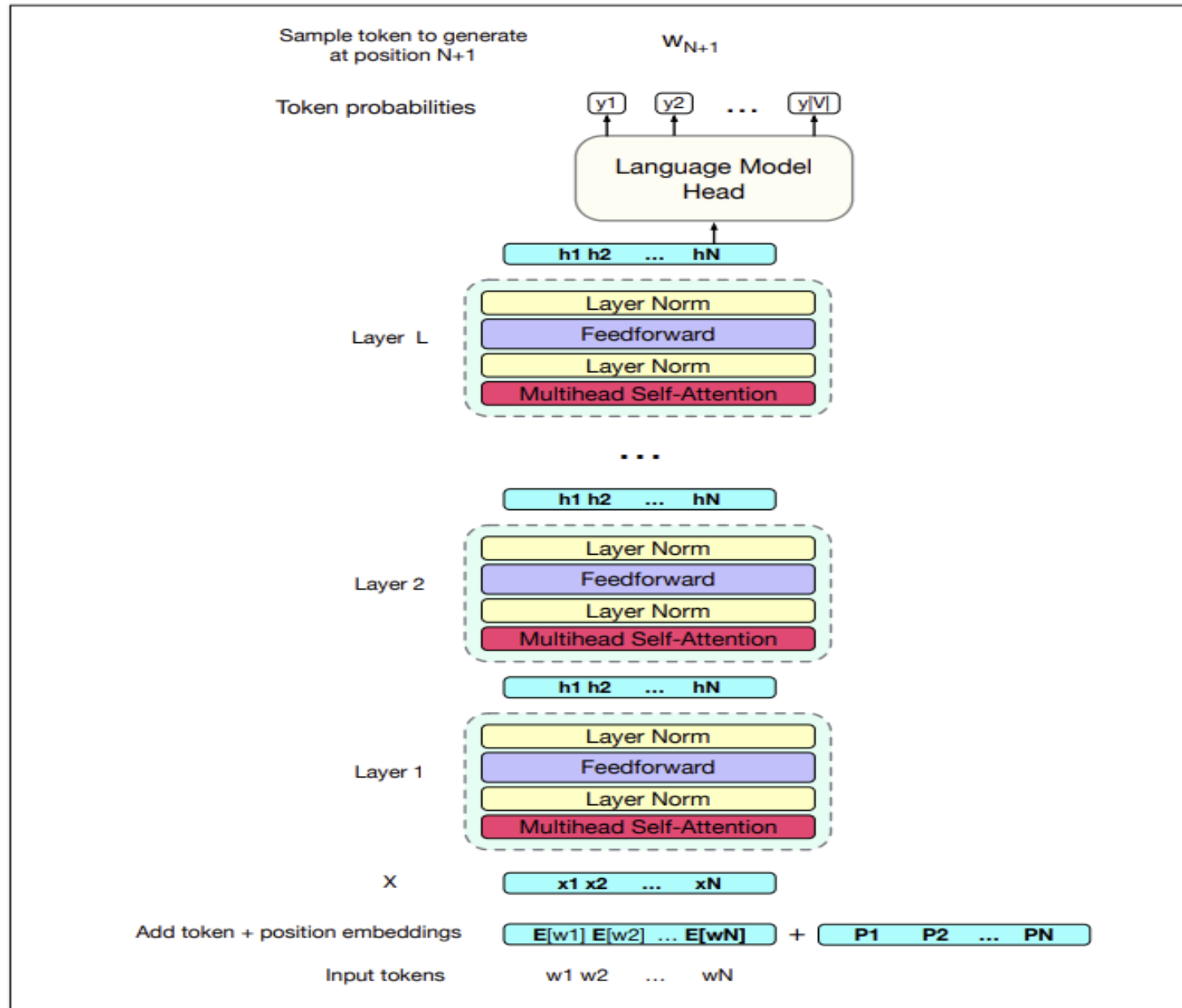
$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3 \quad (10.30)$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5) \quad (10.31)$$

# Transformer Blocks



**Figure 10.12** A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.



**Figure 10.14** A final transformer decoder-only model, stacking post-norm transformer blocks and mapping from a set of input tokens  $w_1$  to  $w_N$  to a predicted next word  $w_{N+1}$ .

# Text Summarization

- ▶ Extractive summarization, which selects key sentences from the original text, and abstractive summarization, which generates new sentences to capture the main points.
- ▶ **Extractive Summarization**
- ▶ Text Parsing
  - The LLM first breaks down the text into sentences or phrases
- ▶ Feature Extraction
  - It then analyzes each sentence or phrase, looking for features like word frequency, sentence position, and the presence of keywords.
- ▶ Scoring
  - Each sentence or phrase is assigned a score based on its importance, using statistical or linguistic features.
- ▶ Selection and Aggregation:
  - Finally, the highest-scoring sentences or phrases are selected and combined to form the summary.

# Abstractive Summarization

- ▶ Understanding Context

The LLM analyzes the input text to understand its meaning, relationships between ideas, and overall structure.

- ▶ Generating New Text

It then generates new, concise sentences that capture the key information from the original text.

- ▶ Rephrasing and Paraphrasing

Instead of simply copying sentences, the LLM rephrases and paraphrases the key points to create a coherent and informative summary



# Other approach

- ▶ **Chunking**

If the input text is too long for the LLM's context window, it can be divided into smaller chunks, summarized individually, and then summarized again to create a final summary.

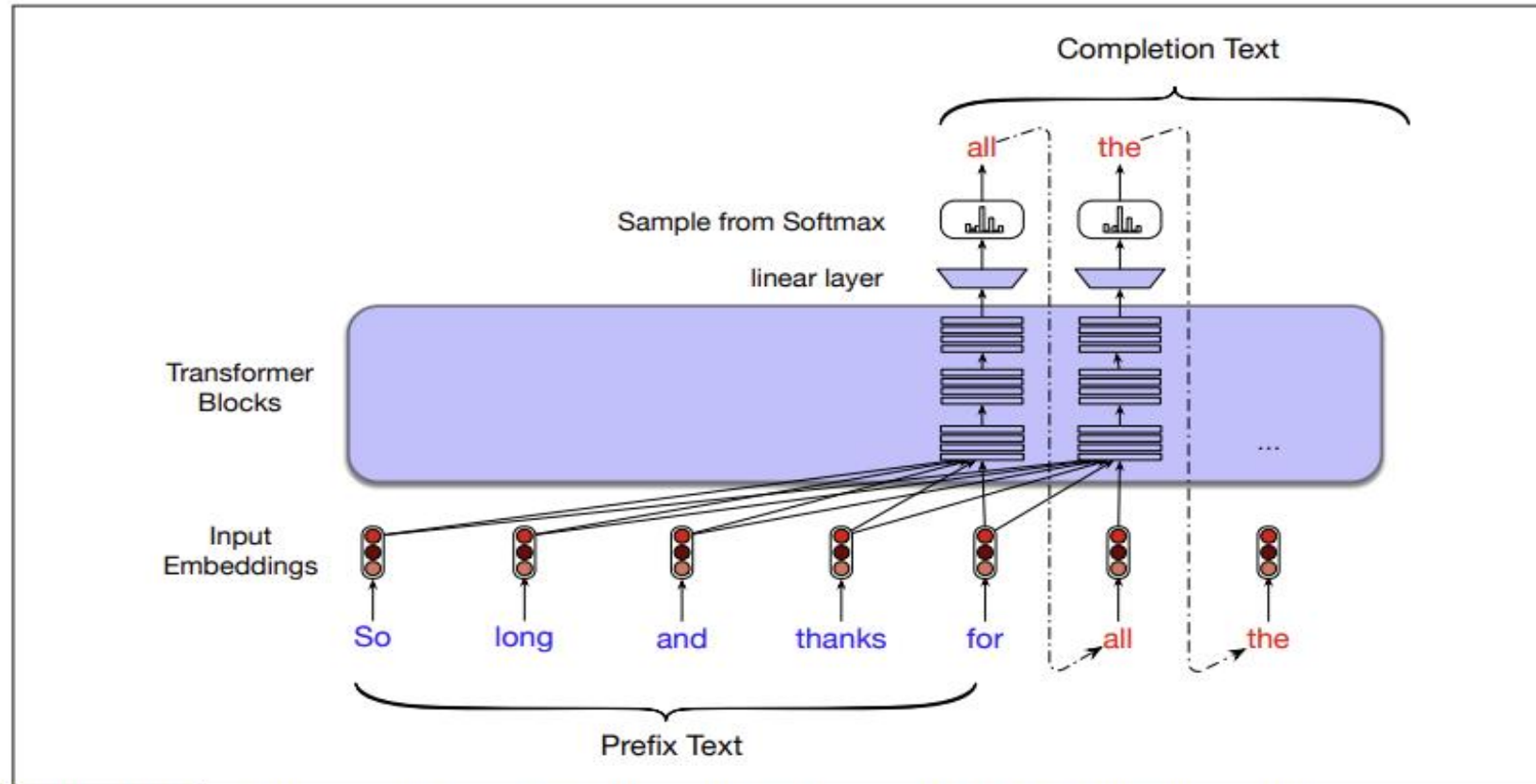
- ▶ **MapReduce**

This technique involves splitting the text into smaller chunks, summarizing each chunk, and then combining the summaries..

- ▶ **Refine**

This approach starts with an initial summary of the first chunk and gradually refines it by adding information from subsequent chunks.

# Transformer-based large language models



**Figure 10.15** Autoregressive text completion with transformer-based large language models.

# Large Language Models with Transformers

- ▶ Conditional generation can even be used to accomplish tasks that must generate longer responses. Consider the task of text summarization, which is to take a long text summarization text, such as a full-length article, and produce an effective shorter summary of it. We can cast summarization as language modeling by giving a large language model a text, and follow the text by a token like **tl ;dr**; this token is short for something like **‘too long; don’t read’** and in recent years people often use this token, especially in informal work emails, when they are going to give a short summary. **We can then do conditional generation: give the language model this prefix, and then as it to generate the following words, one by one, and take the entire response as a summary**

# Large Language Models with Transformers

## Original Article

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. “We will not ship snow to any states in the northeast!” says Waring’s website, ShipSnowYo.com. “We’re in the business of expunging snow!”

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

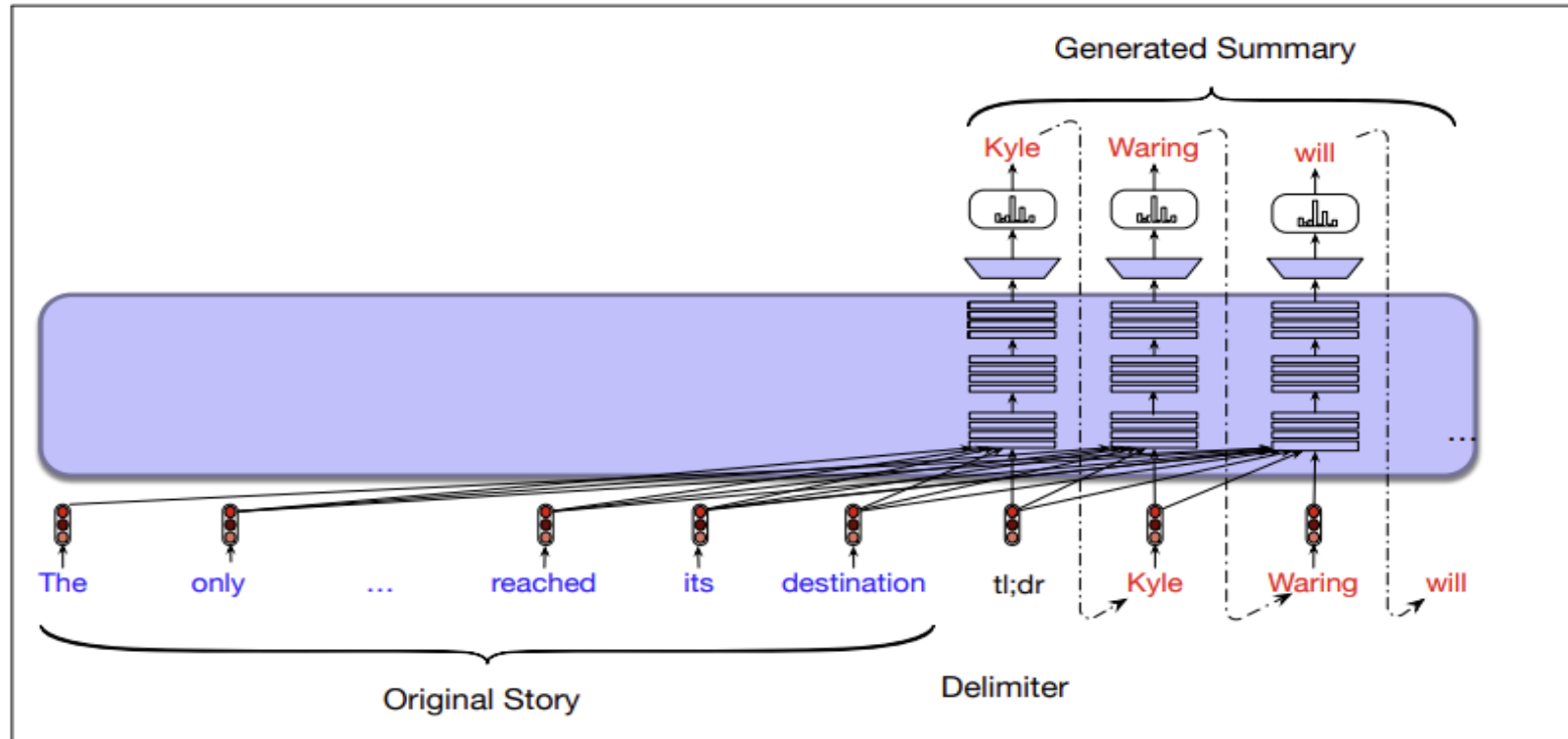
According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. His business slogan: “Our nightmare is your dream!” At first, ShipSnowYo sold snow packed into empty 16.9-ounce water bottles for \$19.99, but the snow usually melted before it reached its destination...

## Summary

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

**Figure 10.16** Examples of articles and summaries from the CNN/Daily Mail corpus ([Hermann et al., 2015b](#)), ([Nallapati et al., 2016](#)).

# Summarization LLM



**Figure 10.17** Summarization with large language models using the `tl;dr` token and context-based autoregressive generation.

# Large Language Models: Generation by Sampling

- ▶ The core of the generation process for large language models is the task of choosing the single word to generate next based on the context and based on the probabilities that the model assigns to possible words. This task of choosing a word to generate based on the model's probabilities is called decoding.
- ▶ Decoding from a language model in a left-to-right manner (or right-to-left for languages like Arabic in which we read from right to left), and thus repeatedly choosing the next word conditioned on our previous choices is called autoregressive generation or causal LM generation.
- ▶ The most common method for decoding in large language models is sampling.
- ▶ sampling from a model's distribution over words means to choose random words according to their probability assigned by the model.

# Random sampling

- ▶ We can formalize this algorithm for generating a sequence of words  $W = w_1, w_2, \dots, w_N$  until we hit the end-of-sequence token, using  $x \sim p(x)$  to mean ‘choose  $x$  by sampling from the distribution  $p(x)$ ’:

```
i ← 1
w_i ~ p(w)
while w_i != EOS
    i ← i + 1
    w_i ~ p(w_i | w_{<i})
```

The algorithm above is called **random sampling**, and it turns out random sampling doesn't work well enough. The problem is that even though random sampling is mostly going to generate sensible, high-probable words, there are many odd, low-probability words in the tail of the distribution.

# Summarization LLM

- ▶ Which words do we generate at each step? One simple way to generate words is to always generate the most likely word given the context. Generating the most likely word given the context is called **greedy decoding**. A greedy algorithm is one that makes a choice that is locally optimal, whether or not it will turn out to have been the best choice with hindsight. Thus in greedy decoding, at each time step in generation, the output  $Y_t$  is chosen by computing the probability for each possible output (every word in the vocabulary) and then choosing the highest probability word (the argmax):

$$\hat{w}_t = \operatorname{argmax}_{w \in V} P(w | \mathbf{w}_{<t}) \quad (10.46)$$



# Summarization LLM

- ▶ In practice, however, we don't use greedy decoding with large language models. A major problem with greedy decoding is that because the words it chooses are (by definition) extremely predictable, the resulting text is generic and often quite repetitive. Indeed, greedy decoding is so predictable that it is deterministic; if the context is identical, and the probabilistic model is the same, greedy decoding will always result in generating exactly the same string.

# Summarization LLM

- ▶ an extension to greedy decoding called **beam search** works well in tasks like machine translation, which are very constrained in that we are always generating a text in one language conditioned on a very specific text in another language. In most other tasks, however, people prefer text which has been generated by more sophisticated methods, called **sampling methods**, that introduce a bit more diversity into the generations.

# Hugging face transformers library



Easy to download and use a state-of-the-art  
NLP model for inference

All models are PyTorch or TensorFlow models

Very simple and straightforward to use

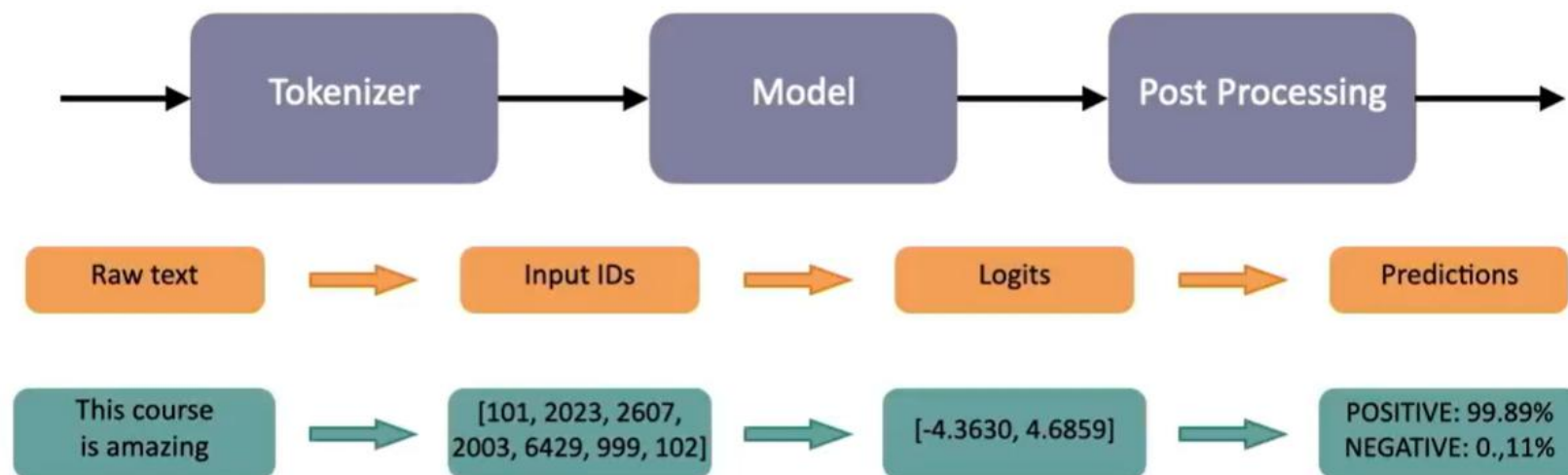
# Hugging face Transformer pipeline



Every pipeline brings together all the steps need to work with a model

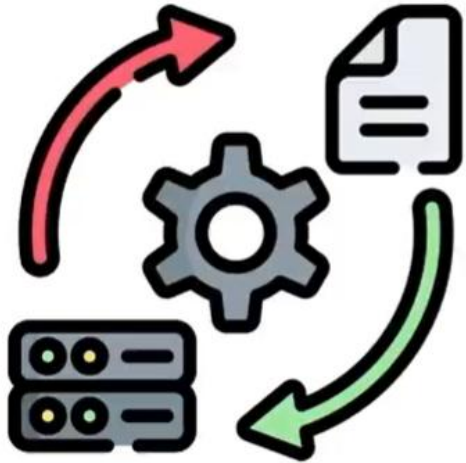
Can access, use, and tweak individual stages in the pipeline as well

# Hugging face Transformer pipeline



Data transformed and flows through the pipeline and model to give predictions

# T5 Model



Text to text transformer model where the input and output are always text strings

Can use the same model, loss function, and hyperparameters on any NLP task

Machine translation, document summarization, question answering

Trained using a new pretrained dataset C4

C4= Colossal, clean, crawled, corpus

# Pegasus model

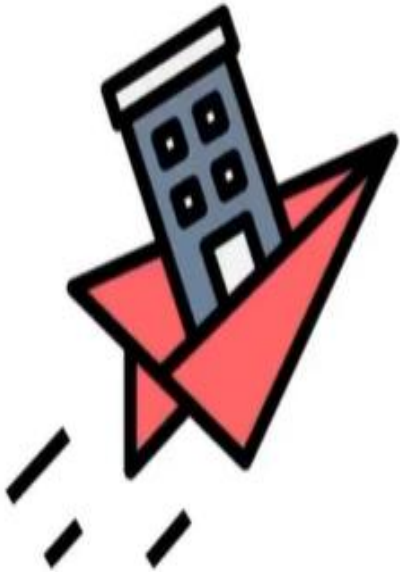


Sequence-to-sequence model using encoder-decoder architecture

Pretraining task is very similar to summarization

Trained using MLM (masked language modeling) and GSG (Gap Sentence Generation)

# BART large CNN



Sequence-to-sequence model using encoder-decoder architecture

Pretrained on English language and fine-tuned on the CNN Daily Mail dataset

Works well for summarization task



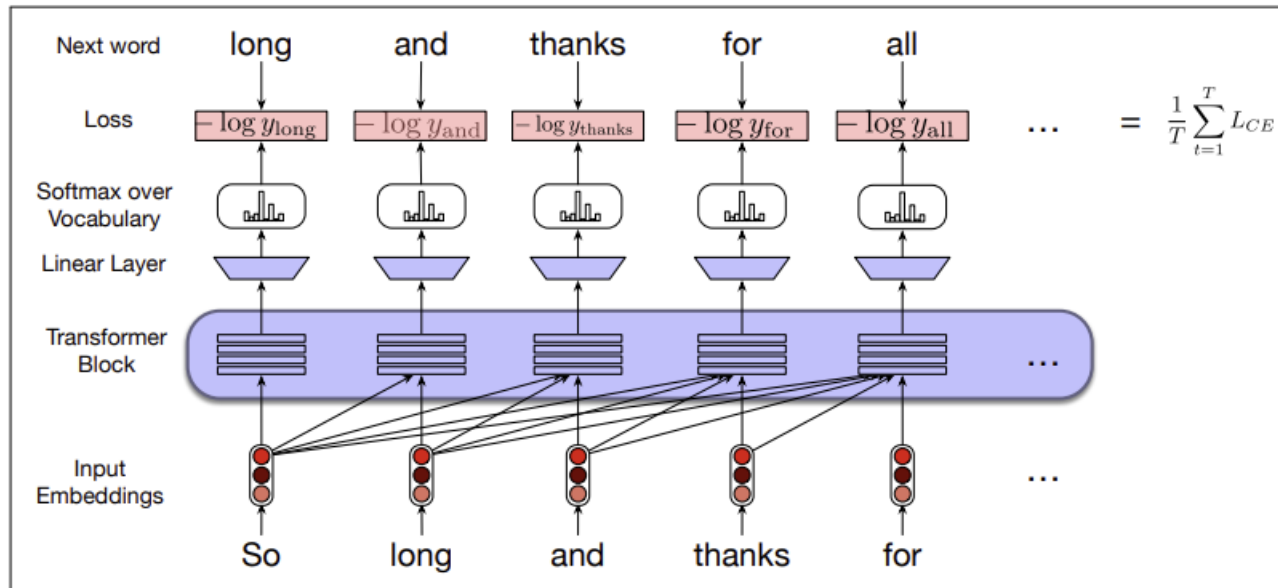
# Large Language Models: Generation by Sampling

- ▶ Nucleus or top-p sampling :
- ▶ One problem with top-k sampling is that  $k$  is fixed, but the shape of the probability distribution over words differs in different contexts. If we set  $k = 10$ , sometimes the top 10 words will be very likely and include most of the probability mass, but other times the probability distribution will be flatter and the top 10 words will only include a small part of the probability mass. top-p sampling An alternative, called top-p sampling or nucleus sampling (Holtzman et al., 2020), is to keep not the top  $k$  words, but the top  $p$  percent of the probability mass. The goal is the same; to truncate the distribution to remove the very unlikely words.
- ▶ Temperature sampling : In temperature sampling, we don't truncate the distribution, but instead reshape it. The intuition for temperature sampling comes from thermodynamics, where a system at a high temperature is very flexible and can explore many possible states, while a system at a lower temperature is likely to explore a subset of lower energy (better) states. In low-temperature sampling, we smoothly increase the probability of the most probable words and decrease the probability of the rare words

# Large Language Models: Training Transformers

- ▶ Self-supervised training algorithm :
- ▶ we take a corpus of text as training material and at each time step  $t$  ask the model to predict the next word. We call such a model self-supervised because we don't have to add any special gold labels to the data; the natural sequence of words is its own supervision! We simply train the model to minimize the error in predicting the true next word in the training sequence, using cross-entropy as the loss function.
- ▶ This idea that we always give the model the correct history sequence to predict the next word (rather than feeding the model its best case from the previous time step) is called **teacher forcing**.

# Training transformer as a language model



**Figure 10.18** Training a transformer as a language model.

# Harms from Language model

- ▶ **problem called hallucination.** Language models are trained to generate text that is predictable and coherent, but the training algorithms we have seen so far don't have any way to enforce that the text that is generated is correct or true. This causes enormous problems for any application where the facts matter!
- ▶ A second source of harm is that language models **can generate toxic language.** Gehman et al. (2020) show that even completely non-toxic prompts can lead large language models to output hate speech and abuse their users. Language models also generate stereotypes (Cheng et al., 2023) and negative attitudes (Brown et al., 2020; Sheng et al., 2019) about many demographic groups

# Causes of LLM Hallucinations

- ▶ Insufficient or Poor Training Data
- ▶ Misalignment Between Training and Real-World Data
- ▶ Overfitting
- ▶ Limited Domain Knowledge
- ▶ Lack of Contextual Understanding

# Overcome Hallucinations

- ▶ Improving Training Data

Enhancing the quality and diversity of training data can help reduce hallucinations.

- ▶ Fine-tuning and Fact-checking

Fine-tuning LLMs on specific tasks and incorporating fact-checking mechanisms can improve accuracy and reliability.

- ▶ Retrieval-Augmented Generation (RAG)

RAG combines LLMs with retrieval systems to allow them to access and utilize external information sources, reducing reliance on internal knowledge and minimizing hallucinations

- ▶ Prompt Engineering

Carefully crafting prompts can help guide LLMs to generate more accurate and relevant responses

# Harms from Language model

- ▶ Language models can also be used by malicious actors for generating text for **misinformation**, phishing, or other socially harmful activities (Brown et al., 2020). McGuffie and Newhouse (2020) **show how large language models generate text that emulates online extremists**.
- ▶ Language models also present **privacy issues** since they can leak information about their training data. It is thus possible for an adversary to extract training-data text from a language model such as an individual person's name, phone number, and address (Henderson et al. 2017, Carlini et al. 2021).
- ▶ Copywrite issue.

# Summary

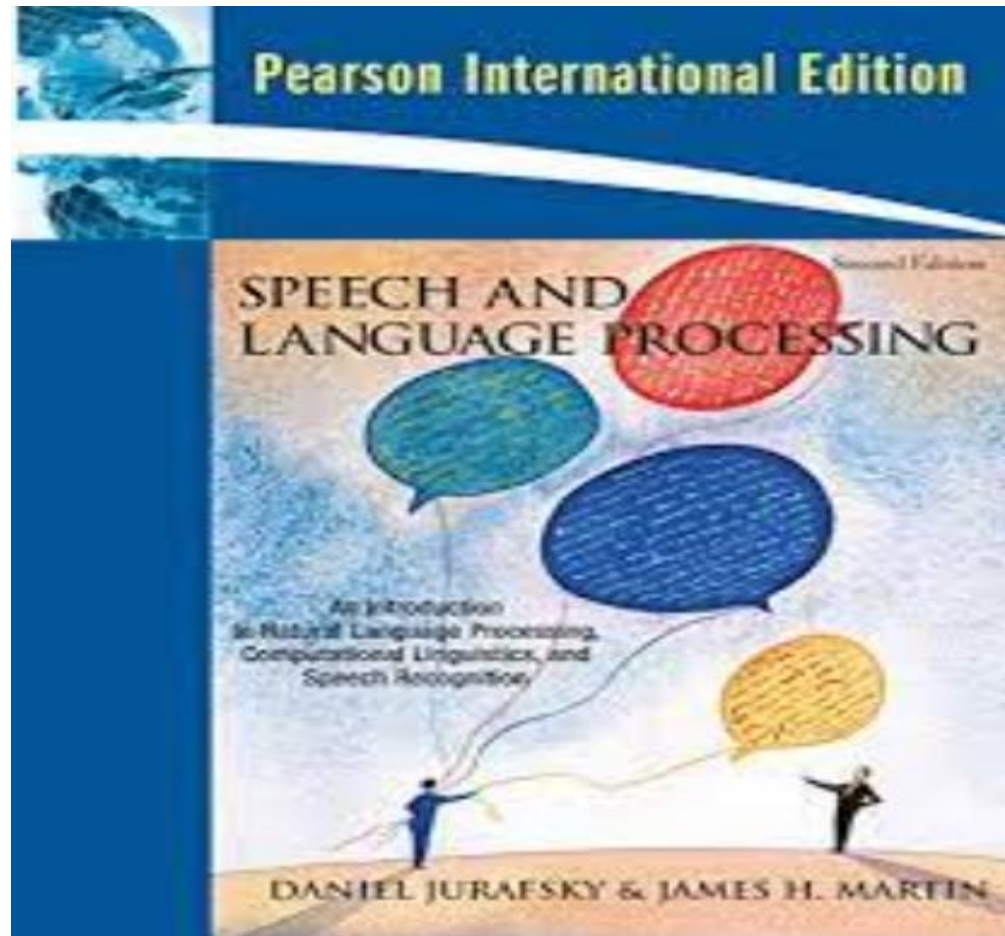
- ▶ Transformers are non-recurrent networks based on self-attention. A self-attention layer maps input sequences to output sequences of the same length, using attention heads that model how the surrounding words are relevant for the processing of the current word.
- ▶ A transformer block consists of a single attention layer followed by a feedforward layer with residual connections and layer normalizations following each. Transformer blocks can be stacked to make deeper and more powerful networks.
- ▶ Language models can be built out of stacks of transformer blocks, with a linear and softmax max layer at the top.
- ▶ Transformer-based language models have a wide context window (as wide as 4096 tokens for current models) allowing them to draw on enormous amounts of context to predict upcoming words



# Summary

- ▶ Many NLP tasks—such as question answering, summarization, sentiment, and machine translation—can be cast as tasks of word prediction and hence addressed with Large language models.
- ▶ The choice of which word to generate in large language models is generally done by using a sampling algorithm.
- ▶ Because of their ability to be used in so many ways, language models also have the potential to cause harms. Some harms include hallucinations, bias, stereotypes, misinformation and propaganda, and violations of privacy and copyright

# Reference



## Chapter 10

# Question



Thank you

